Finance Data Processing & Access Control Backend

System Design · API Documentation · LLD / HLD

Node.js · Express · MongoDB Atlas · JWT · RBAC

Author	Vamshi Kumar Boda
Email	bodavamshikumar2002@gmail.com
Assignment	Finance Dashboard Backend – Zorvyn FinTech
Stack	Node.js · Express · MongoDB Atlas · Mongoose · JWT
Version	1.0.0 | April 2026

1. Project Overview

This document covers the complete technical design for a Finance Dashboard backend system. It serves as both the system design specification and the API reference for reviewers and future developers.

The backend supports multi-role user management, financial record CRUD, aggregated dashboard analytics, and strict access control — built as a clean, layered Node.js/Express service backed by MongoDB Atlas.

1.1 Tech Stack

Layer	Technology	Purpose
Runtime	Node.js 20+	Server-side JavaScript execution
Framework	Express 4.x	HTTP routing, middleware pipeline
Database	MongoDB Atlas (M0 free tier)	Document storage, aggregation pipeline
ODM	Mongoose 8.x	Schema validation, query building
Auth	jsonwebtoken + bcryptjs	JWT signing, password hashing
Validation	Joi / Zod	Request schema validation
Logging	Winston	Structured request/error logging
Env	dotenv	Environment config management

2. High-Level Design (HLD)

2.1 System Architecture

The system follows a layered REST architecture with five tiers:

- Client Layer — web dashboard, mobile app, or any HTTP consumer
- Express API Gateway — JWT auth, RBAC, rate-limiting, request validation
- Service Layer — business logic (Auth, Users, Finance, Analytics)
- Data Layer — Mongoose models accessing MongoDB Atlas collections
- Infrastructure — Atlas cluster, Winston logger, dotenv config

2.2 Architecture Diagram (Text)

HLD — Component Flow

```
┌──────────────────────────────────────────────────┐
│                Client Layer                      │
│   Web App · Mobile App · API Consumers           │
└──────────────────────────────────────────────────┘
                  │  HTTPS / REST + Bearer JWT
                  ▼
┌──────────────────────────────────────────────────┐
│          Express API Gateway (app.js)            │
│  Rate Limit · Auth MW · Role MW · Validate MW    │
└──────────────────────────────────────────────────┘
      │           │            │
  Auth        User Svc     Finance Svc     Analytics
  Svc          (CRUD)      (Records)         Svc
      │           │            │              │
      ▼           ▼            ▼              ▼
┌──────────────────────────────────────────────────┐
│          Mongoose / MongoDB Atlas                │
│    users · financial_records · categories        │
└──────────────────────────────────────────────────┘
```

2.3 Role-Based Access Control

Role	Description	Capabilities
Viewer	Read-only dashboard	GET /dashboard/* only
Analyst	Data analysis	GET /records + all dashboard endpoints
Admin	Full control	All endpoints: CRUD records, manage users

2.4 Request Lifecycle

Middleware pipeline for every authenticated request

```
1. Client → Express router
2. authenticate()  — verify JWT, attach req.user
3. authorize(role) — RBAC check → 403 if denied
4. validate(schema)— Joi/Zod → 400 if invalid
5. Controller      — parse params, call service
6. Service         — business logic, DB query
7. Mongoose        — MongoDB Atlas query
8. apiResponse     — uniform JSON response → client
```

3. Low-Level Design (LLD)

3.1 Folder Structure

Project layout

```
finance-backend/
├── src/
│   ├── config/
│   │   ├── db.js              # MongoDB Atlas connection
│   │   └── env.js             # dotenv loader
│   ├── middleware/
│   │   ├── auth.middleware.js  # JWT verify
│   │   ├── role.middleware.js  # RBAC guard factory
│   │   ├── validate.js         # Joi/Zod wrapper
│   │   └── errorHandler.js     # Global error formatter
│   ├── modules/
│   │   ├── auth/
│   │   │   ├── auth.routes.js
│   │   │   ├── auth.controller.js
│   │   │   └── auth.service.js
│   │   ├── users/
│   │   │   ├── user.routes.js
│   │   │   ├── user.controller.js
│   │   │   ├── user.service.js
│   │   │   └── user.model.js
│   │   ├── records/
│   │   │   ├── record.routes.js
│   │   │   ├── record.controller.js
│   │   │   ├── record.service.js
│   │   │   └── record.model.js
│   │   └── dashboard/
│   │       ├── dashboard.routes.js
│   │       ├── dashboard.controller.js
│   │       └── analytics.service.js
│   └── utils/
│       ├── apiResponse.js
│       ├── logger.js
│       └── pagination.js
├── server.js
├── .env
└── README.md
```

3.2 Module Responsibilities

Module	File(s)	Responsibility
Auth	auth.service.js	bcrypt hash, JWT sign/verify, login logic
Users	user.service.js	CRUD users, role assignment, status toggle
Records	record.service.js	CRUD financial records, filter/paginate
Analytics	analytics.service.js	MongoDB $group/$match aggregations
Middleware	auth/role/validate	JWT, RBAC, Joi validation, error format
Utils	apiResponse.js	Uniform { success, data, meta } envelope

4. Database Schema

4.1 User Model

Field	Type	Constraints / Notes
_id	ObjectId	Auto-generated primary key
name	String	required, trim
email	String	required, unique, lowercase
passwordHash	String	required, bcrypt hash (never plaintext)
role	String enum	viewer | analyst | admin (default: viewer)
status	String enum	active | inactive (default: active)
createdAt	Date	Mongoose timestamps: true
updatedAt	Date	Mongoose timestamps: true

4.2 FinancialRecord Model

Field	Type	Constraints / Notes
_id	ObjectId	Auto-generated primary key
userId	ObjectId (ref: User)	required, foreign key to users
amount	Number	required, > 0
type	String enum	income | expense (required)
category	String	required, e.g. salary, rent, utilities
date	Date	required, transaction date
notes	String	optional description
isDeleted	Boolean	default: false — soft delete flag
createdAt / updatedAt	Date	Mongoose timestamps: true

4.3 Category Model (optional collection)

Field	Type	Notes
_id	ObjectId	Auto-generated
name	String	required, unique — e.g. 'salary', 'rent'
type	String enum	income | expense

4.4 Entity Relationships

- One User → Many FinancialRecords (1:N)
- Each FinancialRecord belongs to exactly one Category (N:1)

- Roles are stored as a string field on the User document (no separate Role collection needed for this scope)

5. API Endpoints

Base URL: `http://localhost:3000/api` | All protected endpoints require: `Authorization: Bearer <jwt_token>`

5.1 Authentication

POST `/api/auth/register` Register a new user

Access: Public — no token required

Request body / params
```
{ "name": "Vamshi", "email": "v@example.com", "password": "Secure@123" }
```

Response (200)
```
{ "success": true, "data": { "_id": "...", "name": "Vamshi", "role": "viewer" } }
```

Note: Default role is viewer. Admin must manually upgrade role via PATCH /users/:id.

POST `/api/auth/login` Login and receive JWT

Access: Public

Request body / params
```
{ "email": "v@example.com", "password": "Secure@123" }
```

Response (200)
```
{ "success": true, "data": { "token": "<jwt>", "user": { "role": "admin" } } }
```

Note: Token expires in 7d by default (configurable via JWT_EXPIRES_IN env var).

GET `/api/auth/me` Get currently authenticated user

Access: Any authenticated role

Request body / params
```
— (no body)
```

Response (200)
```
{ "success": true, "data": { "_id": "...", "name": "...", "role": "analyst" } }
```

5.2 User Management

GET `/api/users?page=1&limit=20` List all users (paginated)

Access: Admin only

Request body / params
```
Query: page (default 1), limit (default 20), role, status
```

Response (200)
```
{ "success": true, "data": [...], "meta": { "total": 42, "page": 1, "pages": 3 } }
```

GET /api/users/:id Get single user by ID

Access: Admin only

Request body / params
```
— URL param :id
```

Response (200)
```
{ "success": true, "data": { "_id": "...", "name": "...", "role": "analyst", "status":
"active" } }
```

POST /api/users Create user with explicit role

Access: Admin only

Request body / params
```
{ "name": "Jane", "email": "j@co.com", "password": "Pass@1", "role": "analyst" }
```

Response (200)
```
{ "success": true, "data": { "_id": "...", "role": "analyst" } }
```

PUT /api/users/:id Update user name, email, or role

Access: Admin only

Request body / params
```
{ "name": "Jane Doe", "role": "admin" }
```

Response (200)
```
{ "success": true, "data": { "_id": "...", "name": "Jane Doe", "role": "admin" } }
```

Note: Role changes take effect on the user's next login (new JWT is issued).

PATCH /api/users/:id/status Toggle active / inactive

Access: Admin only

Request body / params
```
{ "status": "inactive" }
```

Response (200)
```
{ "success": true, "data": { "status": "inactive" } }
```

Note: Inactive users receive 401 on subsequent requests.

DELETE /api/users/:id Delete user

Access: Admin only

Request body / params

```
— URL param :id
```

Response (200)
```
{ "success": true, "data": { "message": "User deleted" } }
```

Note: Hard delete. Consider soft-deleting by setting status=inactive in production.

5.3 Financial Records

GET `/api/records` List records with filters

Access: Admin, Analyst

Request body / params
```
Query params: type (income|expense), category, startDate, endDate, page, limit
```

Response (200)
```
{ "success": true, "data": [...], "meta": { "total": 120, "page": 1 } }
```

Note: All queries auto-filter isDeleted:false.

GET `/api/records/:id` Get a single record

Access: Admin, Analyst

Request body / params
```
— URL param :id
```

Response (200)
```
{ "success": true, "data": { "_id": "...", "amount": 5000, "type": "income", "category":
"salary" } }
```

POST `/api/records` Create a financial record

Access: Admin only

Request body / params
```
{ "amount": 5000, "type": "income", "category": "salary", "date": "2026-03-01", "notes":
"March salary" }
```

Response (200)
```
{ "success": true, "data": { "_id": "...", "amount": 5000 } }
```

Note: amount must be > 0. date must be a valid ISO 8601 date.

PUT `/api/records/:id` Update a record

Access: Admin only

Request body / params
```
{ "amount": 5500, "notes": "Updated" }
```

Response (200)
```
{ "success": true, "data": { "amount": 5500, "updatedAt": "..." } }
```

DELETE `/api/records/:id` Soft-delete a record

Access: Admin only

Request body / params
```
— URL param :id
```

Response (200)
```
{ "success": true, "data": { "message": "Record deleted" } }
```

Note: Sets isDeleted:true. Record remains in DB for audit purposes.

5.4 Dashboard & Analytics

GET `/api/dashboard/summary` Total income, expense, net balance

Access: Admin, Analyst, Viewer

Request body / params
```
Optional query: startDate, endDate
```

Response (200)
```
{ "success": true, "data": { "totalIncome": 85000, "totalExpense": 42000, "netBalance":
43000 } }
```

GET `/api/dashboard/by-category` Totals grouped by category

Access: Admin, Analyst, Viewer

Request body / params
```
Optional query: type (income|expense)
```

Response (200)
```
{ "success": true, "data": [ { "category": "salary", "total": 70000 }, ... ] }
```

Note: Uses MongoDB $group aggregation.

GET `/api/dashboard/trends` Monthly or weekly trend data

Access: Admin, Analyst

Request body / params
```
Query: period (monthly|weekly), year
```

Response (200)
```
{ "success": true, "data": [ { "month": 3, "income": 30000, "expense": 18000 }, ... ] }
```

GET `/api/dashboard/recent` Last N transactions

Access: Admin, Analyst, Viewer

Request body / params
```
Query: limit (default 10)
```

Response (200)
```
{ "success": true, "data": [...] }
```

6. Access Control Matrix

Endpoint	Viewer	Analyst	Admin
GET /dashboard/summary	✓	✓	✓
GET /dashboard/by-category	✓	✓	✓
GET /dashboard/recent	✓	✓	✓
GET /dashboard/trends	✗	✓	✓
GET /records	✗	✓	✓
GET /records/:id	✗	✓	✓
POST /records	✗	✗	✓
PUT /records/:id	✗	✗	✓
DELETE /records/:id	✗	✗	✓
GET /users	✗	✗	✓
POST/PUT/DELETE /users	✗	✗	✓
PATCH /users/:id/status	✗	✗	✓
GET /auth/me	✓	✓	✓

7. Validation & Error Handling

7.1 HTTP Status Codes

Code	Meaning	When used
200	OK	Successful GET, PUT, PATCH, DELETE
201	Created	Successful POST (resource created)
400	Bad Request	Validation failure (Joi/Zod schema rejected)
401	Unauthorized	Missing/expired JWT or inactive user
403	Forbidden	Valid token but insufficient role
404	Not Found	Resource ID does not exist
409	Conflict	Duplicate email on register/create
500	Server Error	Unhandled exceptions (logged internally)

7.2 Standard Error Response Shape

Error envelope

```
{
  "success": false,
  "message": "Validation failed",
  "errors": [
    { "field": "amount", "message": "\"amount\" must be greater than 0" }
  ]
}
```

7.3 Input Validation Rules

- amount: number, required, > 0
- type: string, required, must be 'income' or 'expense'
- category: string, required, min 2 chars
- date: required, valid ISO 8601
- email: valid email format, max 255 chars
- password: min 8 chars, at least one uppercase, one digit
- role: must be one of viewer | analyst | admin
- status: must be active | inactive

8. Key Implementation Patterns

8.1 RBAC Middleware

role.middleware.js

```
const authorize = (...allowedRoles) => (req, res, next) => {
  if (!allowedRoles.includes(req.user.role)) {
    return res.status(403).json({ success: false, message: 'Access denied' });
  }
  next();
};

// Usage
router.post('/records', authenticate, authorize('admin'), createRecord);
router.get('/records',  authenticate, authorize('admin','analyst'), getRecords);
```

8.2 Soft Delete Pattern

record.service.js — deleteRecord

```
const deleteRecord = async (id) => {
  const rec = await Record.findById(id);
  if (!rec) throw new NotFoundError('Record not found');
  rec.isDeleted = true;
  await rec.save();
  return rec;
};

// All list queries include: { isDeleted: false }
```

8.3 Analytics Aggregation

analytics.service.js — getSummary

```
const getSummary = async (filters = {}) => {
  const result = await Record.aggregate([
    { $match: { isDeleted: false, ...filters } },
    { $group: {
        _id: '$type',
        total: { $sum: '$amount' },
        count: { $sum: 1 }
    }}
  ]);
  const income  = result.find(r => r._id === 'income')?. total ?? 0;
  const expense = result.find(r => r._id === 'expense')?.total ?? 0;
  return { totalIncome: income, totalExpense: expense, netBalance: income - expense };
};
```

8.4 Pagination Helper

utils/pagination.js

```
const paginate = (query, { page = 1, limit = 20 }) => ({
  skip: (page - 1) * limit,
  limit: Number(limit),
});

// Returns meta for response
const paginationMeta = (total, page, limit) => ({
  total,
  page: Number(page),
  pages: Math.ceil(total / limit),
  limit: Number(limit),
});
```

9. Environment Configuration

.env.example

```
PORT=3000
MONGODB_URI=mongodb+srv://<user>:<pass>@cluster.mongodb.net/financedb
JWT_SECRET=your_super_secret_key_here
JWT_EXPIRES_IN=7d
NODE_ENV=development
LOG_LEVEL=info
```

9.1 Setup & Run

- Clone the repo and cd into it
- Run: npm install
- Copy .env.example → .env and fill in MONGODB_URI and JWT_SECRET
- Start dev server: npm run dev (nodemon)
- Start prod server: npm start
- Run tests: npm test (Jest + Supertest)

10. Design Decisions & Assumptions

Decision	Rationale
Soft delete (isDeleted flag)	Preserves audit trail; records are never physically removed
Role stored in JWT payload	Avoids a DB lookup on every request; user must re-login after role change
No refresh token (v1)	Keeps scope minimal; noted as a known tradeoff for production hardening
Roles as strings on User doc	Sufficient for 3 fixed roles; a separate Role collection adds overhead without benefit here
Categories as free strings	Flexible for MVP; can be promoted to a managed collection if admin needs governance
MongoDB Atlas free tier (M0)	Zero-cost for assessment; connection string swappable for any Atlas tier in prod
Joi/Zod validation middleware	Centralised validation decouples schema from controllers and gives consistent 400 errors
Winston logger	Structured JSON logs; log level controlled via LOG_LEVEL env var